

学号		成绩	
----	--	----	--



沈阳建筑大学

编译原理上机报告

名 称 编译原理实验二
 学 院 信息与控制工程学院
 专 业 计算机科学与技术
 班 级 计算机 1902 班
 姓 名 蔡艺洵

2021 年 11 月 9 日

一、上机目的

通过设计、编制、调试一个递归下降语法分析程序，实现对词法分析程序所提供的单词序列进行语法检查和结构分析，掌握常用的语法分析方法。通过本实验，应达到以下目标：

- 1、掌握从源程序文件中读取有效字符的方法和产生源程序的内部表示文件的方法。
- 2、掌握词法分析的实现方法。
- 3、上机调试编出的词法分析程序。

二、基本原理和上机步骤

对于本次上机，我们要先知道，词法分析的方法有自顶向下语法分析法和自底向上语法分析法。本次实验实践的是自上而下的分析法。从文法的开始符号出发，根据文法规则正向推导出给定句子的一种方法；或者说，从树根开始，往下构造语法树，直到建立每个叶的分析方法。递归下降分析程序实现思想简单易懂。程序结构和语法产生式有直接的对应关系。因为每个过程表示一个非终结符号的处理，添加语义加工工作比较方便。

递归下降分析程序的实现思想是：识别程序由一组子程序组成。每个子程序对应于一个非终结符号。

每一个子程序的功能是：选择正确的右部，扫描完相应的字。在右部中有非终结符号时，调用该非终结符号对应的子程序来完成。

但是，我们要尽量避免带左递归和回溯的文法（非确定的自上而下分析法），这时需要我们开展处理工作，对左递归和回溯进行消除。因为非确定的自上而下分析法 实际上是一种穷举的试探方法，其分析效率极低，代价很高，在实用的编译程序中是不常用的。

至于左递归和回溯如何消除，这边不再仔细讲解，直接简述方法和思想：

文法的直接左递归消除公式：

直接左递归形式： $U \rightarrow UX|y$ ；

其中： $x, y \in (V_N \cup V_T)^*$ ， y 不以 U 打头。

直接左递归的消除：

$$U \rightarrow yU'$$

$$U' \rightarrow xU'|\epsilon$$

直接左递归的一般形式： $U \rightarrow UX_1|UX_2|\dots|UX_m|y_1|y_2|\dots|y_n$ ；

其中： $x_i \neq \epsilon, y_i$ 都不以 U 打头。

一般形式直接左递归的消除：

$$U \rightarrow y_1 U' | y_2 U' | \dots | y_n U'$$

$$U' \rightarrow x_1 U' | x_2 U' | \dots | x_m U' | \varepsilon$$

回溯的消除的前提是文法不得含有左递归，可提左因子来消除回溯。

在完成了上述先决条件之后，便可进入构造程序的基本框架阶段。

递归下降分析程序的基本架构如下的法：

变量：ch：当前符号；

函数：READ (ch)：读输入串下一符号；

对于每个非终结符号 $U \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$ 处理的方法如下：

```
P (U) {
    if ch ∈ FIRST( $\alpha_1$ ) then P( $\alpha_1$ )
    else if ch ∈ FIRST( $\alpha_2$ ) then P( $\alpha_2$ )
        ...
    else if ch ∈ FIRST( $\alpha_n$ ) then P( $\alpha_n$ )
    else if ch ∈ FOLLOW(U) then return
    else error }
```

对于每个右部 $\alpha = x_1 x_2 \dots x_n$ 的处理架构如下：

```
P( $\alpha$ ) {
    P( $x_1$ );
    P( $x_2$ );
    ... ..
    P( $x_n$ ); }
```

对于右部中的每个符号 x_i ；

如果 x_i 为终结符号：

```
if(ch= a) READ (ch)
Else error
```

如果 x_i 为非终结符号，直接调用相应的过程：p (x_i)。

三、上机结果

程序流程图：

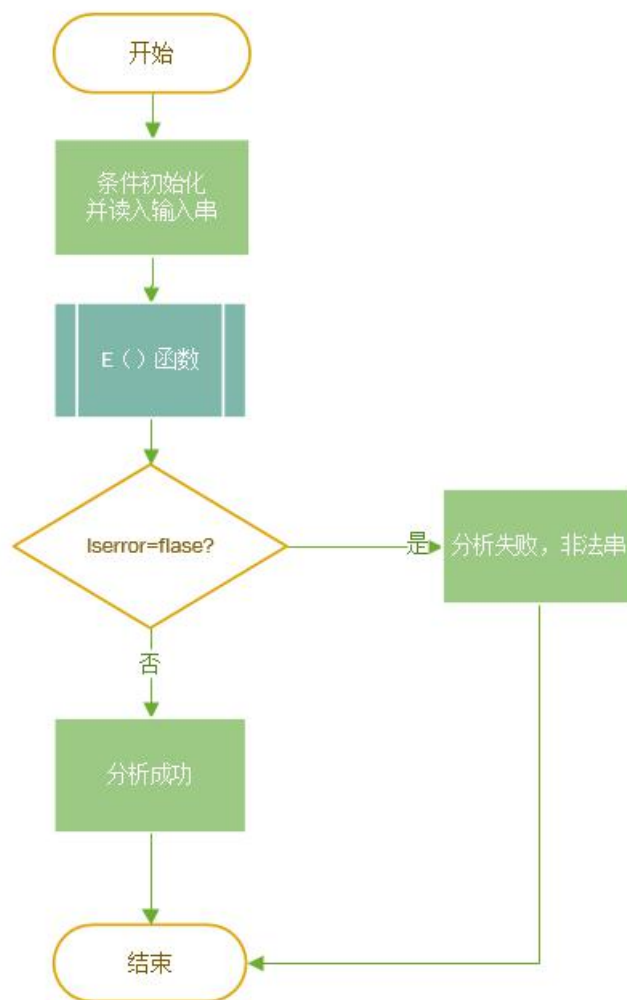


图 1 主流程

剩余各个子流程(子函数)不再仔细展示流程图。

程序输入/输出用例：

对下列文法，用递归下降分析法对任意输入的符号串进行分析：

- (1) $E \rightarrow TG$
- (2) $G \rightarrow +TG$
- (3) $G \rightarrow \epsilon$
- (4) $T \rightarrow FS$
- (5) $S \rightarrow *FS$

(6) $S \rightarrow \varepsilon$

(7) $F \rightarrow (E)$

(8) $F \rightarrow i$

输出的格式如下：

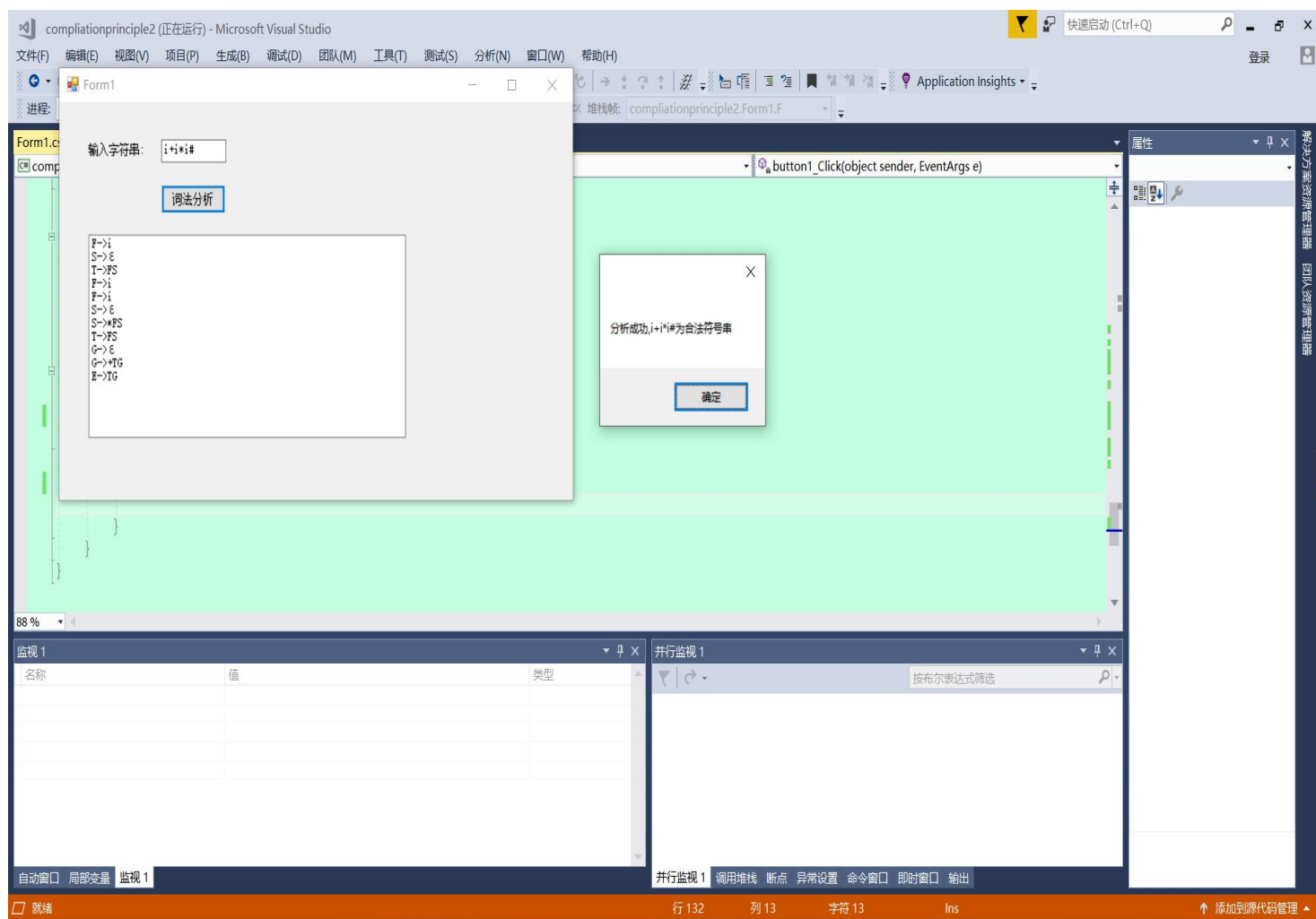
(1)输入一以#结束的符号串(包括+—*/ () i#)：在此位置输入符号串例如：i+i#

(2)输出结果：分析成功，i+i#为合法符号串

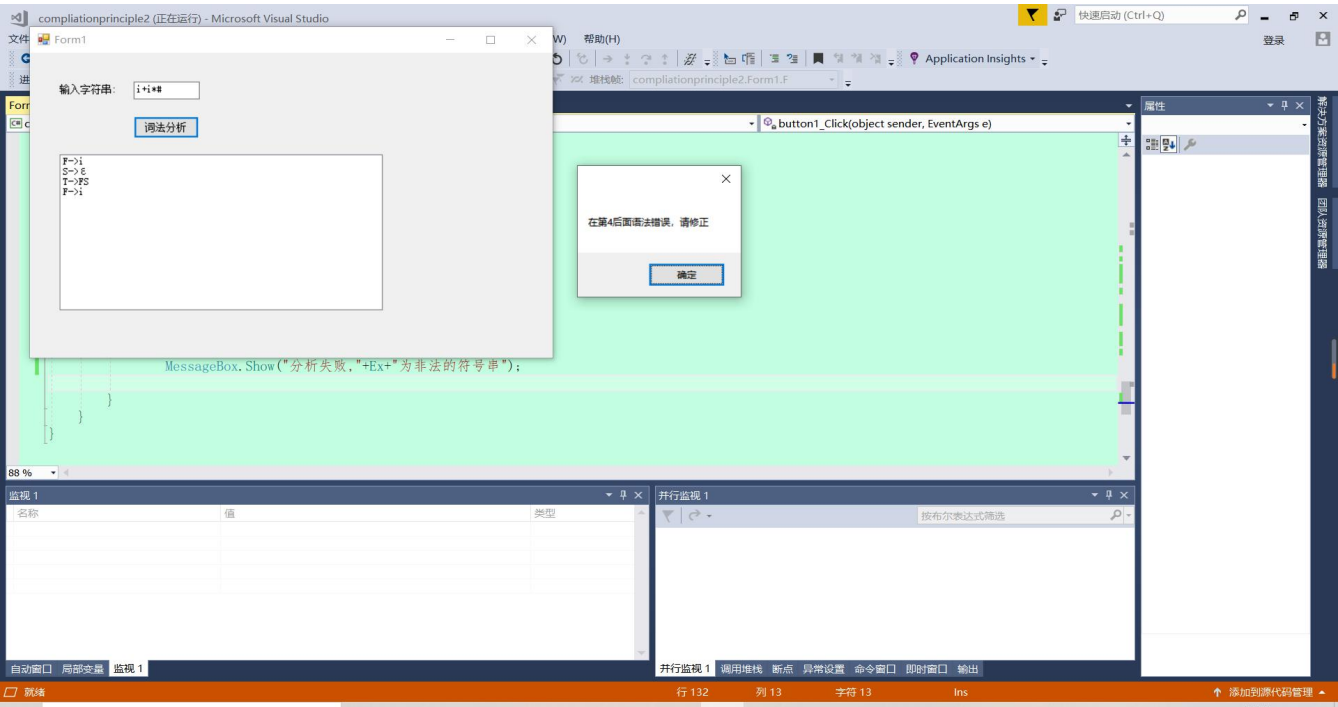
备注：输入一符号串如 i+i#,要求输出为“分析失败，非法的符号串”。

上机结果截图：

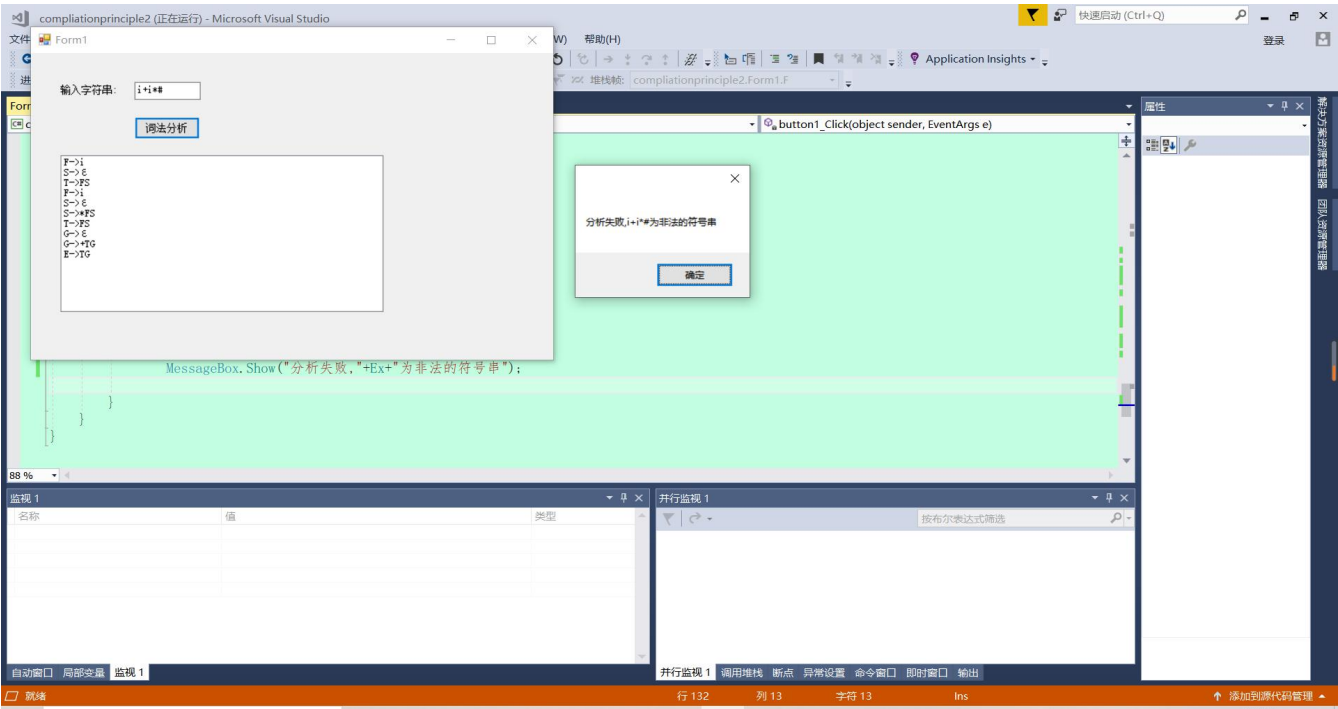
输入 i+i#字符串，结果如下，与预期的一致。



输入 i+i*# 字符串，显示分析错误位置，提示修正，之后显示分析失败，与预计结果一致。



显示“分析失败，i+i*#为非法字符串”



四、讨论与分析

思考题：

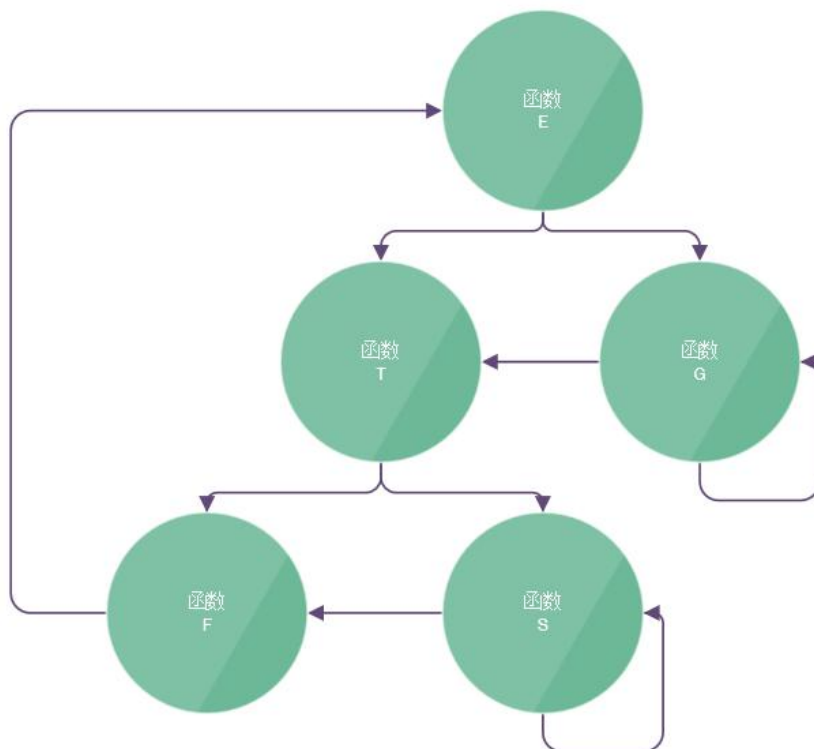
功能描述：本次上机所调试的程序是通过确定的自上而下分析法实现的递归下降语法分析程序。对文法中的每个非终结符编写一个函数（或子程序），每个函数（或子程序）的功能是识别由该非终结符所表示的语法成分。不同函数之间有对应的调用关系，这需要根据文法来确定。通过编写的程序，我们可以对输入的字符串进行识别，看其是否属于该文法下的句子。

程序结构描述：函数调用格式、参数含义、返回值描述、函数功能；函数之间的调用关系图。

本次的函数调用格式无特别，均是示例中的文法非终结符来命名函数名称，函数无参数，无返回值（递归调用），例如文法产生式：（1） $E \rightarrow TG$ ，其函数代码如下图

```
2 个引用  
public void E()  
{  
    T();  
    G();  
    listBox1.Items.Add("E->TG");  
}
```

其余函数也是如此，这里不再一一展示。下面展示各个函数之间的调用关系：



五、源程序清单

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace compilationprinciple2
{
    public partial class Form1 : Form
    {
        char sym;
        int i = 0;
        string Ex;
        bool iserror = false;
        public void Scanner()
        {
            try{
                sym = Ex.ToCharArray()[i];
                if (sym!='#') {
                    i++;}
                return;
            }catch(Exception ex) {
                MessageBox.Show("error:"+ex.Message);
            }
        }
        public void error()
        {
            iserror = true;
            MessageBox.Show("在第"+i+"后面语法错误，请修正");
            return;
        }
        public void E()
        {
            T();
            G();
            listBox1.Items.Add("E->TG");
        }
        public void G()
        {

```



```

        if (sym == '+' )
        {
            Scanner();
            T();
            G();
            listBox1.Items.Add("G->+TG");
        }
        else if ((sym != '#' ) && (sym != ')'))
        {
            error();
        }
        else
            listBox1.Items.Add("G->ε");
    }

    public void T()
    {
        F();
        S();
        listBox1.Items.Add("T->FS");
    }

    public void S()
    {
        if (sym == '*')
        {
            Scanner();
            F();
            S();
            listBox1.Items.Add("S->*FS");
        }
        else if ((sym != ')') && (sym != '#') && (sym != '+'))
        {
            error();
        }
        else
            listBox1.Items.Add("S->ε");
    }

    public void F()
    {
        if (sym == 'i')
        {
            Scanner();
            listBox1.Items.Add("F->i");
        }
    }

```

```

else if (sym == '(')
{

    Scanner();
    E();
    if (sym == ')')
    {
        listBox1.Items.Add("F->(E)");
        Scanner();
    }
    else
        error();
}
else
    error();
}

public Form1()
{
    InitializeComponent();
    //i =0;
}

private void button1_Click(object sender, EventArgs e)
{
    i = 0;
    Ex = txtb1.Text;
    Scanner();
    E();
    if (iserror==false)
    {
        MessageBox.Show("分析成功,"+Ex+"为合法符号串");
    }
    else
        MessageBox.Show("分析失败,"+Ex+"为非法的符号串");
}
}
}

```